PROFESSIONAL ENGINEERING STRATEGY

Geospatial Software Engineer Roadmap

A systematic 26-week curriculum taking a full-stack backend engineer with applied mathematics foundations to owning enterprise farm, forest, and nursery spatial infrastructure compliant with the EU Deforestation Regulation (EUDR).

Blessing Edmund Kwame Dogbe

Software Engineer at Afarinick Company Limited • BSc Applied Mathematics • Creator of Asase GIS

LEAD GEOSPATIAL TRACK

26	8–10	10	EUDR
WEEKS (6 MONTHS)	HOURS / WEEK	PRODUCTION PROJECTS	DEC 2026 MANDATE

1. How to Use This Roadmap

1. **Go through the phases in order:** Each phase builds directly on the preceding one. Do not skip ahead until you can independently deliver that phase's **"Prove it" project**.
2. **Pace & Consistency:** Commit **8–10 hours per week**, designed specifically to balance around a full-time engineering schedule. At this rate, the curriculum spans **26 weeks (6 months)**. Deliverable mastery precedes calendar timelines.
3. **Resource Discipline:** For every phase, focus on **one primary resource** (marked ⭐). Consult secondary materials only when encountering blockers. Build the **"Prove it" project** in a standalone repository before checking off phase completion.
4. **Time Allocation Split:** Dedicate **40% learning**, **50% hands-on building**, and **10% structured notes**. Maintain documentation inside `notes/` (e.g., `01-crs-cheatsheet.md`).
5. **Single Cumulative Anchor Project:** Each phase extends **Asase GIS** (your GeoDjango + PostGIS + Leaflet platform), systematically evolving it into **Asase Trace**—an enterprise cocoa farm-plot compliance and forest monitoring engine.

> ⚠️ **Data Privacy & Security Mandate:** Never commit real Afarinick farmer data, farm boundaries, phone numbers, or community identities to public repositories. All portfolio projects must utilize synthetic polygons, open Ghana benchmarks, or anonymized datasets. Company records remain strictly inside internal systems.

2. Where You're Starting From (Skills Matrix)

TECHNICAL AREA	EXISTING COMPETENCIES (YOU ALREADY HAVE)	TARGET MASTERY GAP TO BRIDGE
Backend	Django, DRF, FastAPI, Pydantic, SQLAlchemy, REST design	GeoDjango internals, OGC Standards (Features/WMS), MVT tile serving
Databases	PostgreSQL, PostGIS basics, relational schema design	Spatial indexing (GiST), EXPLAIN ANALYZE, validity, topology, geography vs geometry
GIS Concepts	GeoJSON, EPSG:4326, basic polygon area in UTM 30N	Datums, Ghana coordinate systems (EPSG:2136/25000), raster data, QGIS mastery
Python Data	Pandas, scikit-learn, NumPy	GeoPandas, Shapely 2.x, pyproj (lat/lon traps), rasterio, xarray/rioxarray
Frontend Maps	Leaflet basic markers & polygons	MapLibre GL JS, vector tile styling, interactive polygon editing (Turf.js)
Remote Sensing	None	Sentinel-2 (10m), NDVI time-series, Google Earth Engine, forest loss datasets
Field Data	None	QField, ODK/KoboToolbox XLSForm, GPS canopy degradation, offline synchronization
DevOps / Cloud	Docker, GitHub Actions, AWS, Terraform	GDAL in multi-stage containers, Cloud Optimized GeoTIFFs (COGs) on S3, Celery async
Mathematics	**BSc Applied Mathematics**	You already possess the geodesy & projection maths. Leverage this core advantage.

> 💡 **Your Strategic Edge:** Most GIS specialists cannot build production backends, and most backend engineers cannot navigate coordinate systems, datums, or geodesics. You are uniquely positioned to bridge both domains, giving Afarinick an indispensable technical asset.

3. Why This Matters at Afarinick: The EUDR Imperative

Afarinick operates at the front line of cocoa rehabilitation, commercial seedling nurseries, and farm mapping across Ghana. The defining regulatory disruption in the cocoa export sector is the **EU Deforestation Regulation (EUDR)**:

📅 **Binding Enforcement Dates:**
• **30 December 2026:** Large & medium enterprise operators.
• **30 June 2027:** Smallholder & micro cooperatives. COCOBOD is currently building the Ghana Cocoa Traceability System.

🌐 **Strict Geolocation Rules:**
• Plots **> 4 ha** require verified **closed polygons**.
• Plots **< 4 ha** require a precise single point.
• Coordinates strictly in **WGS 84 (EPSG:4326)** with **6 decimal places**.

- **Deforestation Cut-off:** Plots must be verified deforestation-free since **31 December 2020** and legal. Farms cannot intersect protected forest reserves unless certified as admitted farms.
- **GeoJSON Specification:** The EU verification platform automatically rejects polygons with self-intersections ("figure-eight" errors), interior holes, slivers, or unclosed coordinate rings.
- **Software Problem:** EUDR compliance is primarily an engineering problem: mobile field capture, polygon validation, spatial indexing, automated satellite cross-checks, and OGC APIs. **This roadmap is engineered around solving it.**

Essential Reference Documentation (Read First – ~1 Hour):
• EU Commission Deforestation-Free Regulation Overview
• EUDR Information System: GeoJSON Specification & Validation Rules
• European Forest Institute: EUDR Preparedness Check of Ghana (2026 Report)

4. 26-Week Master Roadmap at a Glance

PHASE	DURATION	CORE FOCUS AREA	"PROVE IT" PRODUCTION DELIVERABLE
0	Week 0 (4h)	Environment Setup & Orientation	Reproducible local GIS stack (QGIS, Conda/uv, PostGIS Docker)
1	Weeks 1–3	GIS & Geodesy Foundations (QGIS)	Ghana Cocoa Belt cartographic map + Ghana CRS cheat sheet
2	Weeks 4–6	Geospatial Python Stack	`plotcheck` CLI & package: automated EUDR GeoJSON validator
3	Weeks 7–9	Advanced PostGIS & Scale	Asase GIS conflict queries (100k seeded plots, GiST & ST_Subdivide)
4	Weeks 10–13	Remote Sensing & Raster Data	`forestcheck`: automated Sentinel-2 & Hansen deforestation engine
5	Weeks 14–15	Field Data Collection (Mobile)	QField & KoboToolbox XLSForm nursery/farm boundary ingest pipeline
6	Weeks 16–18	Geospatial Backends & Standards	Martin vector tiles (MVT) + OGC API Features + TiTiler COGs
7	Weeks 19–20	Modern Web Mapping (MapLibre)	High-performance React + MapLibre polygon editor & risk dashboard
8	Weeks 21–23	Production Cloud Scale & DevOps	AWS deployment (RDS, S3 COGs, Celery), GeoParquet & H3 indexing
9	Weeks 24–26	Capstone & Enterprise Portfolio	**"Asase Trace" v1.0**: End-to-end production platform + live demo
🌐	Ongoing	Community & Open Source	OSM Ghana contributions, FOSS4G engagement, StackExchange

PHASE 0 Setup and Orientation

Week 0 • ~4 Hours Total

Primary Goal: Install and configure every tool upfront so all subsequent phases focus entirely on spatial algorithms, software engineering, and scientific analysis rather than debugging local installations.

CORE SETUP DELIVERABLES

- [ ] Install **QGIS (LTR version)** from qgis.org/download. Verify GDAL/GEOS/PROJ library versions in Help > About.
- [ ] Create a unified Python geospatial virtual environment with `conda-forge` or `uv` containing: `geopandas shapely pyproj pyogrio rasterio rioxarray xarray pystac-client jupyterlab folium leafmap`
- [ ] Initialize PostGIS via Docker Compose inside Asase GIS; install GDAL command-line utilities (`ogr2ogr` , `ogrinfo` , `gdalinfo`)
- [ ] Register non-commercial accounts on Google Earth Engine and Copernicus Data Space
- [ ] Initialize project repository documentation: create `notes/` in this folder and start `notes/00-glossary.md` to capture spatial terms
- [ ] **Workplace Discovery:** Consult Afarinick's IT and field surveying leads regarding existing GIS file formats (Shapefiles, KML, GeoJSON), mobile software (KoboToolbox, ODK, QField), and coordinate datums in active use.

ESSENTIAL BOOKMARKS FOR DAILY WORKFLOW

GIS StackExchange
gis.stackexchange.com
The primary Q&A knowledge base for spatial queries.

Awesome Geospatial
github.com/.../Awesome-Geospatial
Curated index of modern open-source geo tools.

EPSG Geodetic Registry
epsg.io/32630
Authoritative coordinate systems & datum parameters.

Reference: Ghana Coordinate Reference Systems (CRS) Quick-Guide

EPSG CODE	SYSTEM NAME	DATUM / ELLIPSOID	PRIMARY PRODUCTION USE IN GHANA
`EPSG:4326`	WGS 84 (Geographic)	WGS 84 (Degrees)	Global GPS receivers, mobile apps, GeoJSON format, EUDR compliance portal submissions.
`EPSG:32630`	UTM Zone 30N (Projected)	WGS 84 / Transverse Mercator (Meters)	Standard metric system for 95% of Ghana (Western, Ashanti, Central, Greater Accra). Metric distance & area.
`EPSG:32631`	UTM Zone 31N (Projected)	WGS 84 / Transverse Mercator (Meters)	Far eastern border strips (Volta / Oti). Use geodesic math if working across both zones.
`EPSG:2136`	Accra / Ghana National Grid	War Office / Transverse Mercator (Feet)	Legacy Survey & Mapping Division cadastral maps and historical mining concessions.
`EPSG:25000`	Leigon / Ghana Metre Grid	Clarke 1880 / Transverse Mercator (Meters)	Forestry Commission reserve boundary gazette records and survey pillar networks.

🧭 **Ghana Coordinate Trap:** Never compute surface area using `EPSG:4326` directly. Ghana sits near the equator where 1 degree longitude is ~111.32 km, but planar functions will compute areas in meaningless "square degrees". Always reproject to `EPSG:32630` (UTM 30N) or use ellipsoidal geodesic maths (`ST_Area(geom::geography)`).

PHASE 1 GIS and Geodesy Foundations

Weeks 1–3 • 8–10h / Week

Primary Goal: Think like a professional GIS analyst. Most "GIS bugs" in software are misunderstood spatial concepts: wrong datum, swapped lat/lon, invalid topology, or planar area measured in degrees.

CORE COMPETENCIES TO MASTER

- [ ] Vector vs raster data, spatial layer hierarchies, attribute schemas, and cartographic symbology
- [ ] **Datum vs Projection vs CRS:** Geographic vs projected systems; EPSG registry definitions
- [ ] **Ghana's Coordinate Systems:** `EPSG:4326` (WGS 84), `EPSG:32630` (UTM 30N), `EPSG:2136` (Accra Grid), `EPSG:25000` (Leigon)
- [ ] Why distance and area require projected coordinates or geodesic maths (Ghana sits across UTM 30N/31N)
- [ ] Geometry validity rules: ring orientation (right-hand rule), self-intersections, slivers, gaps, and overlaps
- [ ] File formats: GeoJSON (RFC 7946, lon/lat order), Shapefile limits, **GeoPackage** (modern default), KML, GeoTIFF
- [ ] Spatial operations: buffer, clip, intersect, dissolve, spatial joins, and raster zonal statistics
- [ ] Cartographic composition: printable map layouts, legends, north arrows, dynamic scale bars, choropleths

🇬🇭 GHANA PRACTICE DATASETS TO INTEGRATE

- **HDX:** Ghana administrative boundaries (regions and districts) [data.humdata.org]
- **Geofabrik:** OpenStreetMap road network and settlements extract for Ghana [download.geofabrik.de]
- **Google Open Buildings:** Footprints across Ghanaian agricultural belts [sites.research.google]
- **Global Forest Watch:** Tree-cover loss layers across Western, Ashanti, and Eastern cocoa regions

CURATED LEARNING RESOURCES

⭐ **Spatial Thoughts:** Introduction to QGIS (Complete free course, exercise data, and video lectures)

📖 **Official Manuals:** QGIS Training Manual & QGIS Tutorials by Ujaval Gandhi

📖 **Mathematical Geodesy:** USGS Map Projections: A Working Manual (Snyder) (Read Chapters 1–8)

🎥 **Video Courses:** Matt Forrest: Learn GIS Beginners Course & QGIS v3.40 Training Playlist

📺 **YouTube Channels:** Spatial Thoughts · Klas Karlsson · GeoDelta Labs · GIS Geography

🎯 **PROVE IT DELIVERABLE** Portfolio Project 01

"Ghana Cocoa Belt" Cartographic QGIS Project

Build a complete QGIS project containing administrative districts, OSM roads, forest loss raster tiles, and 20 hand-digitized synthetic cocoa farm parcels.

1. Reproject all layers to UTM 30N (`EPSG:32630`) and calculate true polygon surface areas in hectares.
2. Deliberately construct 3 geometrically invalid polygons (self-intersecting bowtie, unclosed ring, duplicate node) and repair them using *Fix Geometries*.
3. Export an executive map layout to PDF with north arrow, scale bar, legend, and coordinate grid.
4. Document `notes/01-crs-cheatsheet.md` : clear guidance on when to choose EPSG:4326, 32630, and 2136 in Ghana.

PHASE 2 Geospatial Python Stack

Weeks 4–6 • 8–10h / Week

Primary Goal: Do everything you did in QGIS, but in reproducible, testable Python. Automate spatial verification pipelines for enterprise backend services.

CORE COMPETENCIES TO MASTER

- [ ] **GeoPandas Core:** High-performance vector I/O using the `pyogrio` C-engine, coordinate reprojection (`to_crs`), spatial joins (`sjoin`), overlays, dissolutions, clips, and geometric metrics
- [ ] **Shapely 2.x Vectorization:** C-accelerated geometries, `make_valid`, diagnostic analysis with `is_valid_reason`, precision reduction with `set_precision`, binary predicates, and `STRtree` spatial index
- [ ] **pyproj & Geodesy:** Explicit coordinate transforms with `always_xy=True` to eliminate the notorious latitude/longitude axis inversion bug; geodesic area calculations via `Geod(ellps="WGS84")`
- [ ] **GDAL / OGR Command Line:** Format conversions, coordinate transformation pipelines, and spatial queries via `ogr2ogr`, `ogrinfo`, and `gdalinfo`
- [ ] **Real-World Defensive Data Parsing:** Handling corrupted mobile GPS KML exports, uneven CSV coordinate columns, mixed CRS projections, collinear points, and 3D Z-coordinates

CURATED LEARNING RESOURCES

⭐ **Spatial Thoughts:** Python Foundation for Spatial Analysis (Free full course, Jupyter notebooks, and exercises)

📖 **University of Helsinki:** Automating GIS Processes & Introduction to Python for Geographic Data Analysis

📖 **Open Textbooks:** Geocomputation with Python & Qiusheng Wu: Introduction to GIS Programming

📚 **API References:** GeoPandas Guide · Shapely 2.0 Manual · pyproj · pyogrio · GDAL Utilities

🎥 **Video Lectures:** Matt Forrest: Geospatial Python with GeoPandas & Qiusheng Wu: GIS Programming Lectures

📺 **YouTube Channel:** Open Geospatial Solutions (Qiusheng Wu)

🎯 PROVE IT DELIVERABLE — Open-Source Python Tool

`plotcheck`: Automated EUDR Plot-Validator CLI & Python Package

Develop and publish a standalone Python package and CLI utility (with `pytest` suite and GitHub Actions CI) that ingests GeoJSON, KML, GPKG, or CSV files and applies rigorous EUDR validation:

1. **Coordinate Normalization:** Enforces WGS 84 (`EPSG:4326`), strict longitude-before-latitude ordering, and 6 decimal places of precision.
2. **Geometry Sanitation:** Flags and repairs self-intersections (bowties), unclosed rings, interior holes (splits multipolygons per EUDR technical guidelines), duplicate vertices, and micro-slivers.
3. **Area Validation:** Calculates polygon surface area in hectares using both projected UTM 30N and ellipsoidal geodesic algorithms; flags differences exceeding tolerance.
4. **Rule Enforcement:** Automatically enforces the regulatory rule that any parcel exceeding 4.0 hectares must be supplied as a polygon rather than a point coordinate.
5. **Topological Overlap Detection:** Uses a `Shapely STRtree` spatial index to detect intra-dataset farm boundary conflicts in sub-second execution time.
6. **Output Reports:** Emits sanitized EUDR-compliant GeoJSON alongside a structured error report in CSV and HTML.

PHASE 3 Advanced PostGIS at Scale

Weeks 7–9 • 8–10h / Week

Primary Goal: Move from "PostGIS stores my polygons" to "I can design, query, index and tune a spatial database of a million farm plots".

CORE COMPETENCIES TO MASTER

- [ ] **Geometry vs Geography:** Performance trade-offs, planar projection constraints, SRID typing (`geometry(MultiPolygon, 4326)`)
- [ ] **GiST Indexing Mechanics:** R-tree bounding box operations, index scans vs sequential scans, query plan diagnostics with `EXPLAIN (ANALYZE, BUFFERS)` , and the `&&` bounding box operator
- [ ] **Distance Lookups:** Why `ST_DWithin` leverages the spatial index while `ST_Distance() < x` forces expensive table scans
- [ ] **Server-Side Geoprocessing:** `ST_MakeValid` , `ST_IsValidReason` , `ST_SnapToGrid` , and grid-based precision reduction
- [ ] **Handling Complex Geometries:** `ST_Subdivide` to fragment massive national forest reserve polygons, preventing index bottlenecks during spatial joins
- [ ] **Spatial Analytics:** K-Nearest Neighbors via the `<->` index operator; multi-plot self-joins with `ST_Intersection` to compute overlap acreage
- [ ] **Audit Trails:** PostgreSQL trigger functions and historical logging capturing who altered farm boundary vertices and timestamps
- [ ] Bulk data loading: `ogr2ogr` → PostGIS, `shp2pgsql` , high-speed binary `COPY`

CURATED LEARNING RESOURCES

⭐ **Official Workshop:** Introduction to PostGIS (Complete all modules: Indexing, Validity, Geography, KNN, History Tracking)

📖 **PostGIS Documentation:** Official Reference Manual & ST_MakeValid Specification

📖 **Interactive Lab:** Crunchy Data Interactive Browser PostGIS Tutorials

📖 **Performance Optimization:** Paul Ramsey's Engineering Blog (PostGIS Co-founder)

📖 **ORM Integration:** GeoDjango Database API Documentation

📽️ **Video Courses:** Spatial Data Management with PostGIS Playlist (5 Hours) & PostGIS for Newbies

📺 **YouTube Channel:** Crunchy Data Technical Channel

🎯 **PROVE IT DELIVERABLE** — Asase GIS Production Extension

High-Scale Conflict Engine & Spatial Audit Infrastructure

Enhance Asase GIS with enterprise spatial schema and query capabilities:

1. Create a `ProtectedArea` / `ForestReserve` spatial model populated with Ghanaian reserve boundaries.
2. Implement optimized endpoints: `/api/plots/{id}/conflicts/` (detects reserve encroachments and farm overlaps with area in ha) and `/api/plots/nearby/?lat=&lon=&radius_m=` .
3. **Scale Benchmark:** Seed the database with **100,000 synthetic plots**. Document query execution plans using `EXPLAIN ANALYZE` before and after applying GiST indexing and `ST_Subdivide` , recording results in the repository README.
4. Implement PostgreSQL trigger-based temporal audit logging tracking all polygon vertex alterations.
5. Author `docs/queries.sql` : your top 15 annotated production spatial SQL queries.

PHASE 4 Remote Sensing and Raster Data

Weeks 10–13 • 8–10h / Week

Primary Goal: Answer the central technical question of the EUDR: *"Has this plot experienced deforestation since December 31, 2020?"*, and monitor cocoa canopy rehabilitation using Earth observation.

CORE COMPETENCIES TO MASTER

- [ ] **Raster Fundamentals:** Multispectral bands, spatial resolution, nodata masks, bit depths, affine transformation matrices, and CRS reprojection
- [ ] **Satellite Constellations:** Sentinel-2 (10m visual/NIR, 5-day revisit cycle, open access), Landsat archive (30m, 1980s-present); handling tropical cloud cover in Ghana via radar (Sentinel-1 SAR)
- [ ] **Spectral Vegetation Indices:** Computing **NDVI** (Normalized Difference Vegetation Index) & NDMI; scene cloud masking utilizing the Sentinel-2 Scene Classification Layer (SCL)
- [ ] **Python Raster Engineering:** `rasterio` windowed reads and polygon masking; temporal raster datacubes with `rioxarray` and `xarray`
- [ ] **Cloud-Native Geospatial:** Querying SpatioTemporal Asset Catalogs (**STAC**) via `pystac-client` and streaming Cloud Optimized GeoTIFFs (**COG**) over HTTP without full downloads
- [ ] **Google Earth Engine:** Cloud planetary processing using JavaScript Code Editor, Python API, and `geemap`
- [ ] **Regulatory Forest Datasets:** Hansen Global Forest Change (UMD), JRC Global Forest Cover 2020 (EUDR statutory baseline), and FAO Open Foris **Whisp**

CURATED LEARNING RESOURCES

⭐ **Spatial Thoughts:** End-to-End Google Earth Engine (Free full course, code exercises, and video series)

⭐ **Digital Earth Africa:** DE Africa Training Platform & Ghana Sandbox (Free satellite analysis over African agricultural ecosystems)

📖 **Raster Python Guides:** Python for Remote Sensing & Earth Engine and Geemap Free Book

📚 **API References:** rasterio · rioxarray · pystac-client · Cloud-Native Geo Guide

📽️ **Video Lectures:** Earth Engine Basics Video · geemap Python Tutorials

📺 **YouTube Channel:** Digital Earth Africa Technical Series

🎯 **PROVE IT DELIVERABLE** — Automated Analysis Engine

`forestcheck`: Automated EUDR Deforestation Verification Engine

Build an automated analysis module (executable as a standalone CLI or integrated into Asase GIS) that evaluates any input farm polygon against planetary satellite archives:

1. **Baseline Verification:** Intersects parcel geometry with the JRC GFC 2020 raster to determine forest status as of December 31, 2020.
2. **Disturbance Analysis:** Analyzes Hansen Global Forest Change loss layers to quantify post-2021 canopy loss within the parcel boundary.
3. **NDVI Trajectory:** Queries STAC endpoints to extract a monthly Sentinel-2 NDVI time series from 2020 through the present day.
4. **Compliance Score:** Generates an automated risk classification (Low / Medium / High) accompanied by an executive PDF/HTML "Plot Compliance Card".
5. **Scientific Transparency:** Documents sensor limitations (10–30m pixel resolution, cloud artifacts, and agroforestry cocoa vs natural forest spectral similarities).

PHASE 5 Field Data Collection (Mobile GIS)

Weeks 14–15 • 8–10h / Week

Primary Goal: Afarinick's data starts in the field, with officers walking farm boundaries and recording nurseries. Learn how that data is captured, so your backend can accept it cleanly.

- [ ] **QField Ecosystem:** Mobile app for QGIS projects; offline polygon digitizing by walking farm perimeters
- [ ] **ODK / KoboToolbox & XLSForm:** Engineering survey forms with `geopoint`, `geotrace`, and `geoshape` fields
- [ ] **GPS Accuracy:** Phone GPS (±5–15m under cocoa canopy) vs RTK receivers; accuracy threshold filtering and averaging
- [ ] Offline-first sync patterns, conflict resolution strategies, and automated REST API payload ingest into PostGIS

🎯 PROVE IT DELIVERABLE — Mobile Ingest Pipeline

End-to-End Farm/Nursery Field Ingestion Job

Author a production XLSForm survey ("Farm Plot Registration" & "Nursery Batch"), walk an open perimeter using QField / KoboCollect, and build a scheduled backend ingest worker (`manage.py ingest_kobo`) that validates incoming geometry through `plotcheck` before inserting into PostGIS.

PHASE 6 Geospatial Backends and Standards

Weeks 16–18 • 8–10h / Week

Primary Goal: Serve spatial data the way professional systems do: fast vector tiles, standard OGC APIs, and dynamic raster tiles.

- [ ] **GeoDjango Deep Dive:** `GEOSGeometry`, spatial database lookups, transformations, and `django-rest-framework-gis`
- [ ] **OGC API Standards:** Modern RESTful **OGC API – Features** (via `pygeoapi`) replacing legacy WFS
- [ ] **Vector Tile Architecture:** PostGIS `ST_AsMVT`, tile matrix mathematics, lightweight tile servers (**Martin** / `pg_tileserv`), static PMTiles
- [ ] **Dynamic Raster Tile Serving:** Deploying **TiTiler** to serve on-the-fly NDVI map tiles directly from S3 COGs

🎯 PROVE IT DELIVERABLE — Asase GIS Microservices

Containerized Multi-Standard GIS API Suite

Deploy a coordinated Docker Compose environment running Martin tile server delivering 100k plot MVT vector tiles, a pygeoapi instance exposing OGC API Features, and TiTiler serving dynamic Sentinel-2 COG rasters over Asase GIS.

PHASE 7 Modern Web Mapping (MapLibre GL JS)

Weeks 19–20 • 8–10h / Week

Primary Goal: Build map interfaces that field officers and managers actually want to use, and that can display 100k+ plots smoothly.

- [ ] **MapLibre GL JS:** Style specifications, vector tile sources, data-driven styling expressions, and feature-state interactions
- [ ] **In-Browser Geometry Digitization:** Polygon drawing and editing with snapping and instant topology validation
- [ ] **Turf.js Geoprocessing:** Client-side area calculation, intersection detection, and real-time warnings prior to server submission
- [ ] UX design for rural connectivity: clustering, adaptive layer visibility, and mobile-optimized interfaces for Ghana's networks

🎯 PROVE IT DELIVERABLE — Interactive Frontend

React + MapLibre EUDR Interactive Plot Portal

Build a modern dashboard: (1) Vector tile plot layer color-coded by EUDR risk; (2) Attribute inspection panel with live NDVI charts; (3) Interactive polygon drawing tool with client-side Turf.js validation connected to Asase GIS.

PHASE 8 Production, Scale and Cloud-Native

Weeks 21–23 • 8–10h / Week

Primary Goal: Use your DevOps and AWS strengths in a geospatial setting. This is where you can pull ahead of most GIS developers.

- [ ] Building robust, pinned multi-stage Docker containers with official OSGeo GDAL, PROJ, and GEOS binaries
- [ ] Asynchronous background workers (Celery + Redis) for satellite index downloads, tile baking, and heavy spatial analyses
- [ ] Serverless spatial architectures: Hosting Cloud Optimized GeoTIFFs (COGs) and PMTiles on AWS S3 with CloudFront CDN
- [ ] Modern analytical formats: **GeoParquet** and **DuckDB Spatial** for high-speed offline analysis; Uber **H3** hexagonal grids

🎯 PROVE IT DELIVERABLE — AWS Cloud Infrastructure

Terraform-Provisioned Enterprise Cloud Deployment

Deploy Asase GIS on AWS using Terraform (RDS PostgreSQL/PostGIS, S3 + CloudFront for COGs, Celery workers on ECS/EC2), configured with a complete GitHub Actions CI/CD pipeline and nightly automated GeoParquet analytical exports.

PHASE 9 Capstone & Career Launch

Weeks 24–26 • 8–10h / Week

> **Primary Goal:** Combine everything into one polished, documented system: **Asase Trace v1.0**, demonstrating production readiness to Afarinick and the global geospatial community.

ENTERPRISE ARCHITECTURE: ASASE TRACE V1.0

```
1. Mobile Field Capture → QField & KoboCollect XLSForms (Offline Surveys)
2. Ingestion & Validation → plotcheck: Topology normalization, EPSG:4326, 6 decimals, hole splitting
3. Spatial Core DB → PostGIS: GiST indexes, ST_Subdivide reserves, automated audit triggers
4. Satellite Earth Observation → forestcheck: Sentinel-2 NDVI time-series, JRC 2020 baseline, Hansen canopy loss
5. Spatial APIs & Tile Streaming → DRF-GIS, OGC API Features (pygeoapi), Martin MVT, TiTiler COGs
6. Interactive Web Portal → React + MapLibre GL JS: Real-time risk styling, drawing tools, EUDR GeoJSON export
7. Cloud Deployment → AWS (Terraform, RDS PostGIS, S3 CloudFront COGs, Docker CI/CD)
```

CAPSTONE DELIVERABLES CHECKLIST

- [ ] Comprehensive GitHub repository README with architectural diagrams, dataset licenses, CRS rationale, and animated GIF demo
- [ ] 3-minute executive video walkthrough showcasing the platform from field survey ingest to EUDR export
- [ ] Technical engineering article: *"Building an EUDR Plot-Compliance Pipeline for Ghana Cocoa with Open-Source GIS"*
- [ ] Portfolio update: Add a dedicated **"Geospatial Engineering"** domain to `my-portfolio`
- [ ] **Afarinick Lunch-and-Learn:** Present the completed architecture to engineering colleagues and executive leadership

OPTIONAL SPECIALISATION TRACKS (SELECT ONE POST-CAPSTONE)

🤖 Track 1: Geo-AI & Machine Learning
Land-cover classification, tree crown detection, and shaded cocoa identification using TorchGeo, Segment-Geospatial (SAM), and scikit-learn.

🔌 Track 2: QGIS Plugin Development
Build internal Python desktop plugins for Afarinick's field survey and GIS planning teams using the PyQGIS Developer Cookbook.

🚚 Track 3: Spatial Routing & Logistics
Optimize nursery-to-farm seedling delivery routes over Ghana's rural road network using pgRouting and OpenStreetMap topology.

🌍 Track 4: Cloud Big Open Data
Query planetary-scale spatial datasets using Overture Maps, DuckDB spatial, and the Microsoft Planetary Computer.

10 Concepts You Must Explain Without Notes

Before declaring senior proficiency in geospatial engineering, you must be capable of articulately explaining each of these core concepts out loud in under 2 minutes:

01 Datum vs Ellipsoid vs Projection vs CRS
Why GPS points (WGS 84 ellipsoid) do not align with historical Ghana National Grid (War Office ellipsoid) cadastral maps without explicit 7-parameter Helmert transformations.

02 The Planar Area Trap in EPSG:4326
Why running `ST_Area` on EPSG:4326 geometries outputs meaningless square degrees, and the three valid alternatives (casting to geography, reprojecting to UTM 30N, or geodesic maths).

03 Coordinate Axis Ordering Trap
Why RFC 7946 GeoJSON dictates (Longitude, Latitude / X, Y) while Leaflet, GPS hardware, and older legacy standards invert to (Latitude, Longitude / Y, X).

04 Geometry Validity & ST_MakeValid
What constitutes an invalid polygon (self-intersection bowties, duplicate nodes, unclosed rings), and how `make_valid` can silently convert a polygon into a MultiPolygon.

05 GiST Spatial Indexing Mechanics
How R-tree bounding box hierarchies function inside PostgreSQL, and why bounding-box pre-filtering allows `ST_DWithin` to vastly outperform `ST_Distance`.

06 Vector Tiles vs GeoJSON vs WMS
When to serve pre-rendered PNG tiles (WMS), raw vector payloads (GeoJSON), or GPU-styled client vector tiles (MVT) for million-feature datasets.

07 Cloud Optimized GeoTIFFs (COG) & STAC
How HTTP GET Range requests allow reading specific internal image tiles and overviews from S3 without downloading multi-gigabyte satellite scenes.

08 Spectral Limits of NDVI in Agroforestry
Why standard optical vegetation indices cannot reliably distinguish shaded cocoa canopy from natural rainforest, and the necessary role of radar (SAR) and time series.

09 EUDR Technical Geolocation Rules
The exact legal requirements: 4.0 hectare threshold, 6 decimal place precision, December 31, 2020 cut-off date, and strict zero-hole polygon geometry rules.

10 Spatial Resolution vs Accuracy vs Precision
Distinguishing between consumer smartphone GPS drift (±5–15m under canopy), Sentinel-2 raster pixel sizes (10m), and numeric coordinate precision.

Ongoing: Community & Open Source

- [ ] **OpenStreetMap Ghana:** Contribute agricultural and infrastructure mapping via HOT Tasking Manager and query via Overpass Turbo. [Ghana on OSM]
- [ ] **YouthMappers & OSGeo:** Connect with regional Ghanaian university chapters and international open-source geospatial foundations
- [ ] **FOSS4G Conference:** Review technical talks from global and regional FOSS4G conferences; prepare an abstract to present within 12 months
- [ ] **GIS StackExchange:** Answer at least one technical question weekly to refine communication and deep-dive edge cases
- [ ] **Open Source Contributions:** Submit at least one documentation fix or bug patch to `geopandas`, `django-rest-framework-gis`, or `pygeoapi`
- [ ] **Follow Industry Leaders:** Spatial Thoughts · Paul Ramsey · Matt Forrest · Qiusheng Wu · Hatari Labs · HOT OSM

Weekly Operating Routine

```
Mon-Thu (~1.5 h/day): Study primary (⭐) resource → Implement working code in personal sandbox repo
Sat (~3.0 h):         Architect and build the phase "Prove it" project with test coverage
Sun (~15 min):        Write notes/NN-phase.md → Document 3 insights, 1 technical question, 1 limitation
```

🎯 **Three-Question Quality Gate Filter (Before Advancing to Next Phase):**

1. *Can I rebuild this phase's core deliverable from an empty directory without consulting tutorial documentation?*
2. *Does the project possess automated unit tests, a comprehensive README, and explicit CRS documentation?*
3. *Could I clearly explain this system and its operational limits to an executive or non-technical colleague at Afarinick?*

Roadmap Execution Progress Log

PHASE	DATE STARTED	DATE FINISHED	REPOSITORY / DELIVERABLE URL	VERIFICATION NOTES & OBSERVATIONS
0: Setup				QGIS LTR, Conda/uv, PostGIS Docker
1: Foundations				Ghana Cocoa Belt map & CRS cheat sheet
2: Python				`plotcheck` CLI validator package
3: PostGIS				100k plot GiST benchmarks & audit trails
4: Remote Sensing				`forestcheck` Sentinel-2 & Hansen engine
5: Field Data				XLSForm survey & automated ingest worker
6: Backends				Martin vector tiles & OGC API Features
7: Web Mapping				React + MapLibre GL JS interactive portal
8: Production				Terraform AWS deployment & GeoParquet
9: Capstone				**Asase Trace v1.0** production release

Superseding Notice: This 26-week curriculum completely supersedes the generic 12-week outline previously archived in `Geospatial_Software_Engineering_Roadmap.pdf`. Having already developed foundational GeoDjango and Leaflet capabilities through Asase GIS, this roadmap builds directly on existing proficiencies to achieve enterprise-grade mastery.